VoiceOps 2026

A grounded, conversational and voice-enabled support agent for ZentroMart

Participant	Ishaan Agrawal
Repository	Not supplied - replace before submission
Reference date	20 October 2026

1. Project overview

I built one support agent in four stages: a basic LLM baseline, document-grounded retrieval, stateful conversations with verified tools, and an end-to-end Gnani voice layer. My main design goal was correctness: the agent should locate the current policy, admit when the knowledge base is silent, and never claim an action that the code did not complete.

Customer audio	**Gnani STT**	**RAG + LLM + tools**	**Gnani TTS**	**Spoken reply**

Dataset and constraints

I used the supplied synthetic dataset: 21 knowledge documents, seven operational CSV tables, 52 practice questions, and 23 conversations containing 96 turns. I preserve IDs as strings and use 20 October 2026 for all time calculations. Gnani is the only speech service used.

Core principle: retrieve evidence first, expose order details only after verification, then answer briefly in the customer's language.

2. From baseline to grounded answers

Step 1 - baseline

I first call the LLM with only ZentroMart's role and response rules. This deliberately measures what happens before the model sees company documents. The baseline can sound confident while guessing company-specific windows, fees, and timelines, which makes it a useful comparison rather than a safe production agent.

Step 2 - hybrid RAG

I split Markdown files at each section ID and retain document type, status, version date, title, and authority. I index product and pincode rows as structured chunks. Retrieval combines a local BM25-style score with optional embeddings, boosts exact SKUs, limits catalog crowding, and always exposes the policy-precedence rule to the answer model.

Evaluation	Result	What it measures
Step 1 baseline	Not run	LLM without company documents
Step 2 grounded	Not run	Decision and required-fact checks
Retrieval top 8	92.6%	Mean gold-evidence recall
Retrieval hit rate	100.0%	At least one gold source found
Step 3 conversations	Not run	96 turns across 23 conversations

API-dependent results are shown as 'Not run' until I execute evaluate.py with my own OpenAI key. I do not invent evaluation scores. The offline retrieval figures are reproducible without credentials.

Evaluation method

- I compare each decision with the allowed decision labels.
- I check required facts and reject forbidden claims.
- I calculate gold-evidence recall from preserved source IDs.
- Practice reference answers are never provided to the runtime agent.

3. Conversation, tools and voice

Conversation state

I keep the current order ID, verification status, verified order data, failure count, and recent messages inside one conversation. This lets the agent understand follow-ups such as 'How long will that take?' and return to an earlier order after a general-policy question.

Identity boundary

- I request the order ID before any specific lookup.
- The first lookup is unverified and cannot return order facts.
- I ask for one proof: phone last four digits or delivery pincode.
- I allow one retry, then refuse to share private order details.
- I redact verification values and sensitive fields from traces and prompts.

Order and handover tools

I load the supplied lookup_order and create_ticket implementations directly from the dataset. A handover is confirmed only after create_ticket returns created=True. Ticket summaries contain the issue, intent, order ID when known, checks performed, and the escalation reason, without verification secrets.

Gnani voice layer

Customer audio is transcribed through Gnani STT, passed to the same Step 3 agent, and synthesized through Gnani TTS. My default settings are en-IN, Kaveri, and timbre-v2.5. The batch runner accepts a folder and writes a transcript, text reply, WAV reply, retrieved sources, tool trace, and machine-readable result for every clip.

Failure	Recovery
Empty transcript	Ask the customer to repeat; do not call the LLM
No policy evidence	Say I do not have the information; offer a colleague
Verification fails twice	Refuse order details and suggest registered-channel support
Ticket tool fails	State that no ticket was created
Unsupported request	Decline politely without promising an exception

4. Limitations, reproducibility and next steps

Current limitations

- Answer quality still depends on the selected LLM and its structured-output reliability.
- Embedding creation and voice testing require personal API credentials that are never stored in the repository.
- The supplied verification check is an educational mock and is not production authentication.
- The dataset contains text practice cases, so I must record my own consenting voice for the final demo.

What I learned

The strongest retrieval improvement came from respecting document authority and preventing several similar product rows from pushing policy sections out of the top results. Exact SKU matching is useful, but the product row must be paired with the governing policy section.

What I would improve

With more time, I would add a learned multilingual reranker, stream STT and TTS for lower latency, run adversarial privacy tests, and add observability for retrieval confidence, latency, API failures, and tool outcomes. I would also replace the educational four-digit check with production-grade authentication before handling real customer data.

Reproducibility

The repository provides one environment template, pinned dependency ranges, an offline validator, unit tests, a text chat, a batch voice command, and evaluation scripts. Each voice result saves the exact response, evidence IDs, and tool trace. Secrets, the dataset, virtual environments, and generated ticket files are excluded from Git.

Command	Purpose
python validate_submission.py	Validate dataset, privacy gate, tests, and required files
python evaluate.py --suite all	Produce Step 1, Step 2, retrieval, and conversation results
python run.py --input-audio input_audio --output outputs	Run the required end-to-end audio batch
python make_report.py --evaluation-dir evaluation_outputs	Regenerate this report with real scores

Conclusion: I built a support agent that combines grounded evidence, conversation memory, privacy-aware tools, and Gnani speech in one reproducible pipeline. Its most important behavior is knowing when it may answer, when it must ask, and when it must hand over.